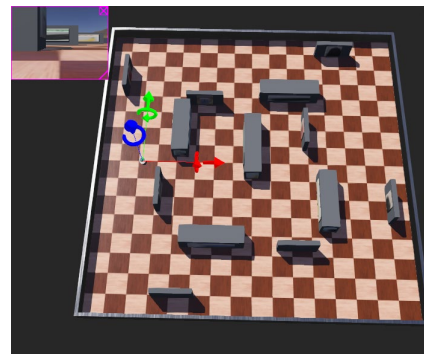
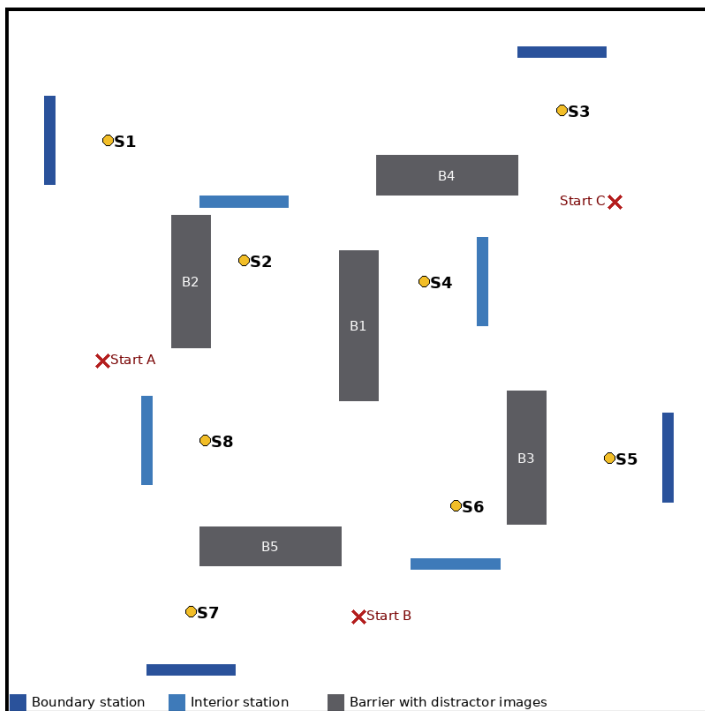


Warning - Copyright Notice

All materials related to this assignment are the intellectual property of Griffith University and are protected by copyright law. **Students are strictly prohibited from uploading, sharing, or publishing any part of this assignment to any public online platforms.** Griffith University reserves the right to investigate and take appropriate action against any student found to have breached this policy.

GROUP PROJECT - Vision-Guided Autonomous Search and Navigation in Webots

Assessment type	Group project
Group size	3 students, unless otherwise approved by the Course Convenor
Due date / weighting	Refer to Canvas
Required platform	Webots R2025a with Python; use the provided project world and materials
Assessment components	Final Report: 35 marks; Oral Presentation: 25 marks (60 marks total)



In webots, you may need to select:
View -> Change View -> Top View

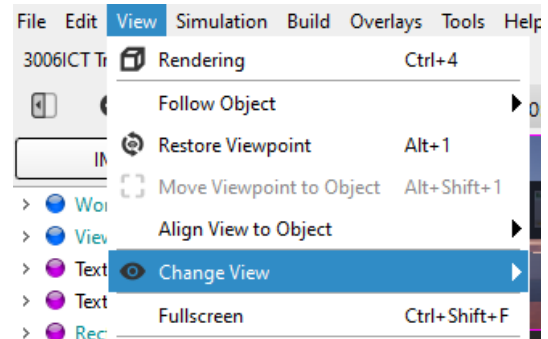


Figure 1. Provided Webots environment with boundary and interior observation stations.

Project goal: Develop one autonomous e-puck controller that uses the onboard RGB camera to determine which observation station displays a specified target, then navigates safely to that station's designated observation position.

1. Project Overview

This open-ended group project brings together concepts developed across the course in a realistic Webots robotics task. You are provided with a Webots environment, an e-puck robot, target reference images, a map, station information, robot pose access, supporting environment data and starter materials. Your group is responsible for designing, implementing and evaluating one autonomous controller that can solve the supplied missions without mission-specific code changes.

The package provides an occupancy-grid map of the environment and access to the e-puck's current position

and orientation through the available robot sensors. A complete visual SLAM implementation is therefore not required. These resources may be used for localisation and navigation, but they do not tell the robot which observation station contains the required target in the current mission.

2. Provided Webots Environment

Materials package: 3006ICT_group_project_webots	
Primary world	worlds/training_start_A.wbt; training worlds for Starts A, B and C are supplied
Arena	4 m × 4 m e-puck-scale environment
Robot	Cyberbotics e-puck with RGB camera, proximity sensors ps0-ps7, GPS and InertialUnit
Observation stations	8 textured barrier stations: 4 boundary stations (S1, S3, S5, S7) and 4 interior stations (S2, S4, S6, S8)
Possible starts	A, B and C
Scene objects	Five navigation barriers B1-B5 with non-target distractor images on all four vertical faces, plus eight textured observation-station barriers
Occupancy grid	maps/occupancy_grid.npy and maps/occupancy_grid.csv; 40 × 40 cells, 0.10 m/cell; 0 = free and 1 = obstacle
Project configuration	config/project_config.json provides the designated starts, station information and observation-position coordinates
Target references	targets/target_reference.png and the supplied target texture images
Target input interface	config/assessment_mission.json; the provided shared controller reads MISSION["target"] at startup

Keep the supplied folder structure unchanged. The observation-station posters are local PNG textures referenced by relative paths from the Webots project.

The project is conducted in a provided Webots environment using an e-puck mobile robot. The environment has been prepared so that you can focus on designing, integrating and evaluating your autonomous solution rather than constructing the simulation environment from scratch.

The provided project resources include:

- **Webots world files (.wbt)** containing the robot, obstacles, observation stations, visual targets, and designated starting positions.
- An **e-puck robot** equipped with:
 - an RGB camera;
 - proximity sensors (ps0–ps7);
 - GPS, which can provide the robot's current position;
 - an InertialUnit, which can provide orientation information.
- An occupancy-grid map of the environment (maps/occupancy_grid.npy and maps/occupancy_grid.csv).
- The observation-station information and designated observation-position coordinates (provided in config/project_config.json).
- A set of reference target images corresponding to the eight possible mission targets.
- Non-target visual images placed on barriers B1–B5, which act as **visual distractors**.
- Designated robot **starting positions**.
- A mission-specific target identity stored in config/assessment_mission.json. The provided controller already reads this file at startup, so students do not need to change the controller code between missions.

A complete visual SLAM system is not required because the environment map and access to the robot's current pose are already available. Your group should decide whether and how these resources are used. Importantly, the provided map and pose information do not reveal which station contains the requested target.

Practical Tips

The following points may help you when developing and testing your system:

- You may use the provided map, GPS and orientation information for localisation and navigation if they are useful for your solution.
- The RGB camera must be used to determine which observation station displays the required target. Map, GPS or station-coordinate information must not be used as a substitute for visual target identification.
- The proximity sensors may provide useful information when the robot is operating near obstacles.
- The observation stations are distributed across both the **boundary and interior** of the environment.
- Images displayed on B1 - B5 are **not valid mission targets**. Your vision system should therefore be able to distinguish the required target from other visual content in the environment.
- The target may appear at a different observation station in different assessment missions. Do not assume a fixed target-to-station relationship.
- You should test the same controller from different designated starting positions and with different target identities. A single successful run is not sufficient evidence of robustness.
- All supplied .wbt files use the same controller. The target identity for the current mission is stored in config/assessment_mission.json. The provided controller already loads this file using `MISSION = json.loads((ROOT / "config" / "assessment_mission.json").read_text());` the required target identity is available from `MISSION["target"]`. The instructor will configure the mission file before an assessment run. Your submitted controller must run unchanged across missions. Open the selected .wbt file and run the simulation; do not edit the controller or mission configuration after the mission is revealed.
- You may use any appropriate combination of the provided information and robot sensors. There is no prescribed navigation, planning, vision or control method, and you are not required to visit the observation stations in any particular order.

3. Mission

Each assessment mission uses a supplied Webots world together with config/assessment_mission.json. The selected .wbt file determines the robot's starting pose and target-to-station arrangement, while config/assessment_mission.json specifies the target object that the robot must find. The provided shared controller already reads the target identity from this file at startup. Before an assessment run, the instructor will configure the required target identity. Students only need to open and run the selected .wbt file; the controller and mission configuration must not be modified during the run.

The robot is given the target identity, but it is not given the station ID or target location. It must use the onboard RGB camera to determine which observation station displays the requested target. After identifying the correct station, the robot must autonomously navigate to that station's designated observation position and stop there.

For assessment, the target station is considered reached when the centre of the e-puck is within 0.20 m (Euclidean distance in the x-y plane) of the designated observation position. The mission must be completed without manual steering and without collision. The target-to-station assignment may change between assessment missions, so a fixed target-to-station mapping will not work.

Mission time limit. Each assessed mission must be completed within 4:00 minutes of Webots simulation time, measured from the start of the simulation. If the success criteria have not been met by 4:00, the mission is recorded as incomplete. Pausing, restarting or resetting the mission is not permitted after the run begins, except where the instructor authorises a restart because of a genuine technical problem.

4. Observation Stations and Visual Targets

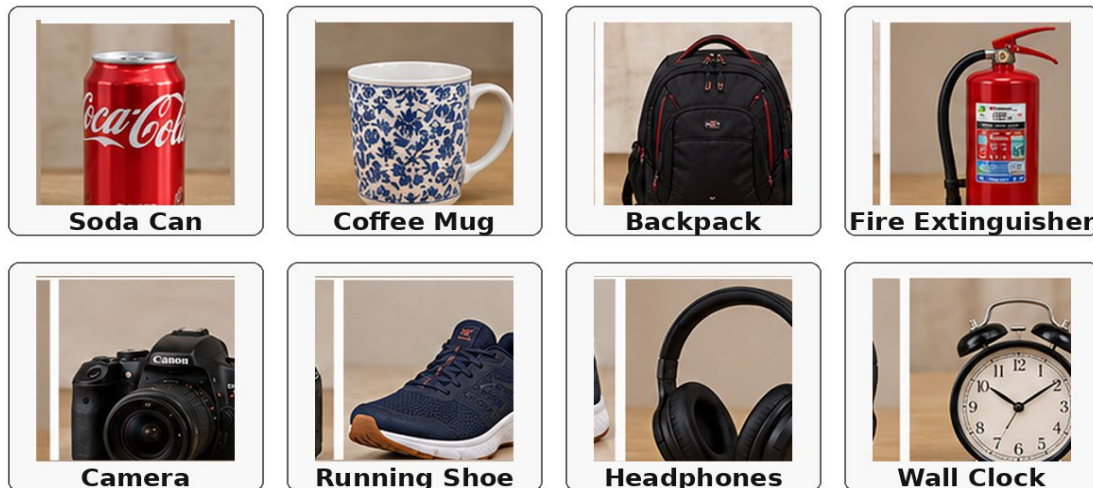


Figure 2. Realistic visual target images used at the observation stations.

Each observation station is a physical barrier carrying one realistic target image. Four stations are located near the environment boundary and four are located within the interior. The eight possible target identities are: soda_can, coffee_mug, backpack, fire_extinguisher, camera, running_shoe, headphones and wall_clock. Barriers B1-B5 display non-target distractor images on their vertical faces; these images are never valid mission targets. The locations of the stations and their designated observation positions are known project resources, but the target-to-station assignment for an assessment mission is unknown to the controller. Target identification must therefore be based on the e-puck RGB camera rather than simulator-provided ground-truth recognition information.

Example mission	training_start_A.wbt: Start A; target = soda_can
Known to the system	Environment map; station and observation-position coordinates; access to current robot pose; required target identity
Unknown to the system	Which observation station displays the required target in the current mission
Autonomy	The same submitted controller must run unchanged; after Run is pressed, no manual steering or mission-specific controller edits are permitted
Mission success	The robot identifies the correct target using the RGB camera, reaches the associated designated observation position, and stops. At the end of the mission, the e-puck centre must be within 0.20 m of that observation position, with no collision or manual intervention.
Target input	Specified in config/assessment_mission.json; the provided shared controller reads the target identity automatically at startup
Mission time limit	Maximum 4:00 minutes of Webots simulation time from the start of the run

5. System Design and Solution Freedom

This is an open-ended project. Your group is responsible for designing and developing the complete autonomous solution to the project task.

There is no prescribed system architecture, computer-vision method, search strategy, planning algorithm, control method or software structure. Different groups may develop substantially different solutions, provided that the final system satisfies the project requirements.

You are expected to identify the technical challenges, determine an appropriate approach, integrate the necessary components, justify your design decisions, and evaluate the completed system. Methods and concepts introduced in the course may be useful, but you are not required to follow any particular solution demonstrated in lectures or workshops. Using a particular algorithm or model does not by itself determine

project complexity; complexity is judged from the technical scope, integration, robustness and quality of the completed system.

6. Project Task and Requirements

Your task is to develop one autonomous vision-enabled controller that can solve multiple missions using the same submitted code. The task is not to drive the robot to a target coordinate that is already known. Instead, the robot is given a target identity and must use its RGB camera to determine which observation station displays that target. It must then navigate safely to the designated observation position associated with that station and stop there.

- After the simulation is started, the system must operate autonomously; no manual steering or keyboard control is permitted.
- The e-puck RGB camera must be used to determine which observation station displays the required visual target in the presence of non-target distractor images.
- The same submitted controller must operate from the designated starting locations without mission-specific code changes.
- The system must remain functional when the target-to-station assignment changes between missions.
- The robot must avoid contact/collision with arena walls, B1-B5 barriers and observation-station structures during an assessed mission.
- After the correct target station has been identified, the robot must reach the associated designated observation position and stop there.
- The solution must not rely on a hard-coded target-to-station mapping, a pre-recorded route for a specific mission, or a mission-specific sequence of timed wheel commands.
- Webots built-in Camera Recognition, simulator ground-truth object identity, or equivalent ground-truth recognition information must not be used to identify the visual target.
- The controller must not read or parse the .wbt file, station texture URLs/ilenames, instructor ground-truth files, or other simulator/project metadata to infer the target-to-station assignment. The station containing the required target must be determined from RGB camera observations.
- Each assessed mission must satisfy the mission success criteria within the 4:00-minute mission time limit.
- The official assessment world geometry, station locations, target arrangement and target assets must not be modified. You may make copies of the training worlds for your own development and testing.
- The submitted system must be reproducible using the provided project environment and the files submitted by the group, without relying on machine-specific absolute file paths.

Provided resources. The project package provides the Webots environment, e-puck robot, observation-station and observation-position coordinates, target reference images, occupancy-grid map, access to robot pose information, robot sensors and supporting environment data. These resources may be used where appropriate. The only information that must be obtained visually is which station displays the required mission target.

7. Assessment Components and Deliverables

Final Report (35 marks)

The final report should present the complete project clearly and professionally. It should explain the problem and project idea, justify the main design decisions, describe the technical approach, implementation and system integration, and present systematic experimental evaluation and discussion. The evaluation should cover the provided three student missions (i.e., worlds/*.wbt), report success/failure and completion time, and include meaningful failure cases where applicable. Include a brief contribution statement or table showing each group member's responsibilities and how the work was integrated. Discuss limitations and possible improvements. The report structure is determined by the group and does not need to follow a prescribed technical decomposition.

Oral Presentation (25 marks)

Each group has a total of 20 minutes for the Oral Presentation: 10 minutes for the presentation, up to 4 minutes for the live Webots demonstration, and 6 minutes for Q&A. The presentation should clearly explain the task, the group's solution, important design decisions and the main experimental results. The live demonstration will use the submitted controller on one instructor-selected assessment mission. The same 4:00-minute mission time limit applies, and the controller must not be edited after the mission is selected. All

group members should understand the complete system and be prepared to answer questions about the design, implementation, experiments, limitations and individual contributions.

Supporting submission. Submit the final report, complete source code/controller, and all additional local files required to reproduce and demonstrate the system through Canvas. If your solution depends on external libraries or pretrained models, provide clear setup/loading instructions and acknowledge them appropriately.

Individual contribution and marks. Where group members make broadly equal contributions, they will normally receive the same group-project mark. If a student's contribution is demonstrably lower than that of the group, that student's individual project mark may be reduced proportionally. A student who makes no meaningful contribution may receive zero for the group project. Evidence may include the contribution statement, source-code/version history, task records, submitted files, presentation and Q&A performance, and other evidence requested by the Course Convenor. The "missing evidence of teamwork and individual contributions" deduction in the report rubric is separate from any individual mark adjustment for unequal contribution.

8. Marking Rubric

Please refer to the submission page for the detailed Marking Rubric.

Final Report (35 marks)

Criterion	Marks
Problem understanding, project idea and design rationale	8
Technical approach, implementation and system integration	9
Experimental evaluation, results and discussion	8
Project complexity and robustness	6
Overall writing quality and report presentation	4
Missing evidence of teamwork and individual contributions	-3
Poor format deduction	-2

Oral Presentation (25 marks)

Criterion	Marks
Clarity and technical explanation	7
Live code/system demonstration	10
Q&A and individual understanding	8

9. Submission and Demonstration Requirements

- Submit the Final Report (pdf or docx). The report must be uploaded separately and **must not** be placed inside the ZIP file.
- The complete reproducible project/code package in zip. **Do not include unnecessary environment folders, downloaded model weights, or large temporary files.** Your code should run **using reasonable relative file paths**. The marker should not need to edit multiple hard-coded absolute paths to run your program.
 - Keep the supplied project folder structure unchanged where possible. Any additional files required by your implementation must be included or clearly documented.
 - The demonstration must use the submitted controller/code. Before the run, the instructor will select the assessment .wbt file and configure the required target identity in config/assessment_mission.json. The provided controller reads this target automatically. Once the selected mission is started, no controller or mission-configuration edits, manual steering, keyboard driving, pausing, restarting or resetting are permitted, except where the instructor authorises a restart because of a genuine technical problem.

- For the live demonstration, the instructor will select one supplied assessment mission. The same submitted controller must be used unchanged, and the mission must be completed within the 4:00-minute mission time limit.
- **All group members** must be prepared to explain the complete project and answer questions about the whole system, not only the component they primarily implemented.

10. Academic Integrity and Use of External Resources

You may use standard libraries and appropriately acknowledged external resources or pretrained models where relevant. Your group must understand and be able to explain the submitted system. Clearly acknowledge external code, models, datasets and other resources. The core project idea, implementation decisions, integration and evaluation should reflect the group's own work. Any use of generative AI tools must comply with the course instructions and Griffith University guidance.